STM32 CAN Telemetry Node

Rev A - Firmware Starter Plan

Author	Joshua Nwaneli
Date	June 2026
Design Tool	Altium Designer
PCB	STM32 CAN Telemetry Node Rev A
MCU	STM32F103C8T6, LQFP-48
Status	Schematic complete - PCB routed - DRC clean - Bring-up planned

This document describes the planned starter firmware for the STM32 CAN Telemetry Node Rev A. The goal is to provide a clear firmware direction for board bring-up: initialize GPIO, ADC, timer input, and CAN, then transmit sensor data using a simple CAN frame. This is a firmware plan; implementation and validation are planned future steps.

1. Firmware Goals

- Bring up the STM32F103C8T6 on the custom PCB.
- Verify GPIO output using LED_STATUS.
- Read analog inputs AIN1-AIN4 through the ADC.
- Read digital inputs DIN1 and DIN2 through GPIO.
- Measure FREQ_IN using a timer-capable input pin.
- Transmit a CAN telemetry frame with ID 0x100 every 100 ms.
- Use LED_CAN for transmit activity and LED_ERROR for fault indication.

2. Development Environment

Item	Planned Choice
IDE	STM32CubeIDE
Programmer/debugger	ST-Link through J4 SWD header
MCU	STM32F103C8T6, LQFP-48
Clock source	External 8 MHz crystal planned
CAN bitrate	500 kbps planned
Transmit period	100 ms planned

3. Peripheral Plan

Peripheral	Purpose
GPIO	LEDs, user button, digital inputs, reset/debug-related signals
ADC	Read AIN1-AIN4 analog sensor channels
CAN	Transmit telemetry frame ID 0x100
Timer input	Measure FREQ_IN or wheel-speed signal
SysTick/timer delay	Schedule 100 ms transmit period
SWD	Programming and debugging through J4

4. Signal Assignment Summary

Signal	Firmware Role
ADC_IN0	AIN1 raw ADC reading
ADC_IN1	AIN2 raw ADC reading
ADC_IN2	AIN3 raw ADC reading
ADC_IN3	AIN4 raw ADC reading
DIGITAL_IN1	DIN1 logic input
DIGITAL_IN2	DIN2 logic input
FREQ_TIMER_IN	Frequency/wheel-speed timer capture input
CAN_TX/CAN_RX	CAN transceiver interface
LED_STATUS	Heartbeat/status LED
LED_CAN	Toggles on CAN transmit
LED_ERROR	Error or fault indication
USER_BUTTON	Firmware-controlled user input

5. Main Loop Pseudocode

```
while (1)
{
    read_adc_channels();
    read_digital_inputs();
    measure_frequency_input();
    pack_adc_values_into_can_frame();
    send_can_message(0x100);
    toggle_led_can();
    update_status_led();
    delay_ms(100);
}
```

6. CAN Message Format

CAN ID	DLC	Bytes	Signal	Description
0x100	8	Byte 0-1	AIN1_RAW	Raw ADC value from AIN1
0x100	8	Byte 2-3	AIN2_RAW	Raw ADC value from AIN2
0x100	8	Byte 4-5	AIN3_RAW	Raw ADC value from AIN3
0x100	8	Byte 6-7	AIN4_RAW	Raw ADC value from AIN4

7. Initial Firmware Milestones

1. Create STM32CubeIDE project for STM32F103C8T6.
2. Configure system clock using the external crystal.
3. Configure GPIO for LEDs and user button.
4. Flash LED blink program and verify LED_STATUS.
5. Configure ADC channels and read AIN1-AIN4.
6. Configure CAN at 500 kbps.
7. Transmit ID 0x100 every 100 ms.
8. Verify CAN frame reception using a CAN analyzer.
9. Add error handling and LED_ERROR behavior.

8. Future Firmware Improvements

- Convert raw ADC counts to calibrated sensor units.
- Add CAN receive support for configuration commands.
- Add diagnostic CAN messages.
- Add input fault detection for open/shorted sensors.
- Add frequency-to-speed conversion for wheel-speed input.
- Add bootloader workflow or USB-C debug/programming in a future hardware revision.
- Add SD card data logging if future hardware supports it.